

Combinatorial Decision Making and Optimization

Exercise-Type Reference Sheet

*A map of the exercise types that can appear in the written exam, organized by area,
each with a short worked example and a strategy for approaching it under time pressure.*

Contents

1	Constraint Programming (CP)	2
1.1	Modelling	2
1.2	Propagation	4
1.3	Search	5
2	SAT	6
2.1	Encoding pseudo-Boolean / cardinality constraints	6
2.2	Direct SAT Encoding of a CSP	7
2.3	Solving: Resolution and DPLL	8
3	SMT	9
3.1	Theories and CDCL(T)	9
3.2	Theory of Arrays	10
3.3	Lazy SMT: the abstraction-refinement loop	10
3.4	Clause Learning in SAT/SMT (CDCL / CDCL(T))	11
4	LP / ILP / MIP	12
4.1	Modelling	12
4.2	Simplex, Gomory Cuts, and Branch-and-Bound (numeric)	13
4.3	Duality and Sensitivity	14

1 Constraint Programming (CP)

1.1 Modelling

How to approach it:

- Always structure the answer in three labelled parts: **Variables and Domains**, **Constraints**, **Objective** (if any). Comment each constraint in one line.
- First find the “obvious” set of decision variables (what am I choosing?), then check whether a compact domain exists.
- If comparing two models, always argue in terms of: number of constraints, arity of constraints, and whether a global constraint applies.
- Never forget *implicit* real-world constraints not stated explicitly in the text (e.g., “a resource cannot be used twice at once”).

Type 1 — Basic modelling from scratch

A tournament has 6 teams. Each team plays every other team exactly once, scheduled over 5 match days. Each day can host at most 3 matches (3 fields available). Model the problem: define variables, domains, and constraints.

Variables and Domains. A variable G_{ij} for each pair of teams (i, j) , $1 \leq i < j \leq 6$, representing the day on which the match between team i and team j is played (there are $\binom{6}{2} = 15$ such matches). $D(G_{ij}) = \{1, \dots, 5\}$.

Constraints.

- *Field capacity*: on each day, at most 3 matches can be played.

$$\forall g \in \{1, \dots, 5\} : |\{(i, j) : G_{ij} = g\}| \leq 3$$

- *No team plays twice on the same day* (implicit constraint, not stated literally in the text but required by the physical setting): for each team $k = 1, \dots, 6$,

$$\text{alldifferent}(\{G_{ij} : i = k \text{ or } j = k\})$$

Type 2 — Comparing alternative models

Consider the Golomb Ruler problem: place m marks on a ruler such that the distance between every pair of marks is different, and the ruler length is minimum. A naive model uses variables $X_1, \dots, X_m$ (mark positions) with constraint: for all $i_1 < j_1, i_2 < j_2, (i_1, j_1) \neq (i_2, j_2)$: $|X_{i_1} - X_{j_1}| \neq |X_{i_2} - X_{j_2}|$. **(i)** Explain why this model is problematic. **(ii)** Propose an improved model using auxiliary variables. **(iii)** Propose a further improvement using symmetry breaking.

(i) With m marks there are $\binom{m}{2} = O(m^2)$ pairwise distances; the naive constraint compares every pair of distances, giving $O(m^4)$ constraints, each *quaternary*. This causes both an excessive

constraint count and weak propagation.

(ii) Introduce auxiliary variables $D_{ij} = |X_i - X_j|$ for every $i < j$ and replace the naive constraint with the global constraint

$$\text{alldifferent}([D_{12}, D_{13}, \dots, D_{(m-1)m}])$$

This drops the constraint count to $O(m^2)$, each ternary, plus one global constraint with dedicated (matching-based) propagation.

(iii) Add the ordering constraint $X_1 < X_2 < \dots < X_m$ (symmetry breaking, removes the $m!$ position permutations). As a side effect: $\text{alldifferent}(X)$ becomes redundant (implied by the strict order), and the absolute value is no longer needed: $D_{ij} = X_j - X_i$.

Type 3 — Global constraints

5 employees must each cover exactly one of 5 distinct shifts in a day (no shift uncovered, no employee double-booked). Express this first using **alldifferent**, then rewrite it in “elementary” form (binary $\neq$ constraints).

Variables. $E_i \in \{1, \dots, 5\}$ for $i = 1, \dots, 5$ (shift assigned to employee i).

Global form: $\text{alldifferent}([E_1, \dots, E_5])$.

Elementary form: $\forall 1 \leq i < j \leq 5 : E_i \neq E_j$ (10 binary constraints, since $\binom{5}{2} = 10$).

Remark. The two forms are semantically equivalent but not computationally equivalent: **alldifferent** achieves GAC through a dedicated matching-based algorithm, while the pairwise decomposition propagates more weakly (each constraint only sees 2 variables at a time).

Type 4 — Implied (redundant) constraint

In the standard N-Queens model (X_i = column of the queen in row i , with $\text{alldifferent}(X)$ and anti-diagonal constraint $\forall i < j : |X_i - X_j| \neq |i - j|$), propose an implied constraint that does not change the solution set but may help propagation. Justify.

The anti-diagonal constraint hides two diagonal directions behind the absolute value: $X_i - i$ constant along a $\searrow$ diagonal, $X_i + i$ constant along a $\swarrow$ diagonal. Introduce

$$A_i = X_i - i, \quad B_i = X_i + i \quad (i = 1, \dots, n)$$

and add $\text{alldifferent}([A_1, \dots, A_n])$ and $\text{alldifferent}([B_1, \dots, B_n])$.

Redundant: these add no new solutions excluded beyond what the anti-diagonal constraint already forbids. **Useful:** **alldifferent** propagates globally (GAC via matching) instead of through $n(n-1)/2$ independent pairwise checks — semantically redundant, computationally significant.

Type 5 — Symmetry breaking

In the Dual Model of N-Queens (Boolean matrix B_{ij} , with $\sum_j B_{ij} = 1$ per row/column), the board has 8 symmetries (rotations/reflections). Propose a symmetry-breaking constraint and explain what it imposes.

Flatten the 2D matrix B into a single sequence of variables. Each of the 7 non-trivial symmetric configurations corresponds to a variable permutation π of the original solution. Impose, for each such permutation:

$$\text{lex} \leq (B, \pi(B)) \quad \text{for all } \pi$$

This forces the chosen solution to be lexicographically no greater than each of its symmetric counterparts, keeping exactly one representative per symmetry class and pruning the search space accordingly.

1.2 Propagation

How to approach it:

- Recall the definition: $C(X_1, \dots, X_k)$ is **GAC** iff every value $v \in D(X_i)$ belongs to at least one *support* (an allowed tuple in C).
- To propagate by hand: for each variable, try each remaining domain value and check whether it can be extended to a full support using the current domains of the other variables. If not, remove it.
- Iterate until a fixpoint (no domain changes) — propagation may need several passes, since removing a value can trigger new removals elsewhere.
- State clearly which consistency level you are achieving (AC for binary constraints, GAC in general, or bound-consistency if only bounds are checked).

Type 1 — Manual domain propagation

Let $X, Y, Z \in \{1, \dots, 8\}$, subject to $X + Y + Z = 10$ and $\text{alldifferent}(X, Y, Z)$. Starting from full domains, apply propagation and show the resulting domains.

Bound reasoning on the sum constraint. The minimum value X can take, given $Y, Z \geq 1$, is $10 - 8 - 8 = -6$, so no reduction from the upper side yet; but since $Y, Z \leq 8$ and distinct from X , and all must be ≥ 1 : the maximum a single variable can take is $10 - 1 - 2 = 7$ (leaving the two smallest distinct values, 1 and 2, for the others). So $D(X) = D(Y) = D(Z)$ shrinks from $\{1, \dots, 8\}$ to $\{1, \dots, 7\}$ (value 8 removed, since it would force the other two to sum to 2 with distinct values ≥ 1 , impossible as $1 + 2 = 3 > 2$... concretely: 8 needs partners summing to 2, but the smallest distinct pair is $1 + 2 = 3$, so 8 is infeasible).

Re-check alldifferent: no singleton domains yet, so no further pruning triggered at this stage.

General method to present on the exam: state the support-based definition, then for a chosen value (e.g. $X = 8$) show explicitly why no support exists ($Y + Z = 2$ with $Y \neq Z$, $Y, Z \geq 1$ is impossible), justifying its removal; repeat for the boundary values until the fixpoint is reached.

Type 2 — GAC check on a global constraint

Consider $\text{AtMostOne}(x_1, \dots, x_n)$ encoded in SAT. Explain when unit propagation over the encoding achieves arc-consistency, using the definition of arc-consistent encoding.

An encoding E of a constraint C is *arc-consistent* iff: (1) whenever a partial assignment is inconsistent w.r.t. C , unit propagation on E causes a conflict; (2) otherwise, unit propagation on E discards exactly the values that cannot be extended to a solution of C .

For $\text{AtMostOne}(x_1, \dots, x_n)$: if two variables x_i, x_j are both assigned true, unit propagation must directly derive a conflict; if exactly one x_i is true, unit propagation must directly force every other x_j to false. This property must be checked directly against the chosen encoding (pairwise, sequential, bitwise, ...): pairwise and sequential encodings satisfy it, while some compact encodings (e.g. Full-Adder-based cardinality encodings) generally do *not*.

1.3 Search

How to approach it:

- These are almost always conceptual/comparison questions (quiz-style), not calculations: know the trade-offs of variable/value ordering heuristics (dom, domWdeg, dynamic domain size) and restart strategies.
- Key fact: an improved model can solve faster *even without* changing the search strategy, if it enables stronger propagation and hence a smaller search tree — not because it enumerates more solutions or removes constraints.
- Randomized heuristics pair naturally with restarts (they benefit from re-exploring with a new random order); deterministic heuristics with a fixed value order gain little from restarting, since they tend to redo the same search.

Type 1 — Heuristic comparison

Which search heuristic is least meaningful to use together with geometric restarting in CP? (a) random variable choice + increasing value order; (b) domWdeg + increasing value order; (c) dynamic domain size + random value order; (d) dynamic domain size + increasing value order.

(d). A fully deterministic heuristic (both variable and value choice deterministic) will retrace essentially the same search path after a restart, since nothing in the strategy varies run to run — restarts are most useful when some randomness (in variable or value choice) lets the solver explore a different part of the tree each time.

2 SAT

2.1 Encoding pseudo-Boolean / cardinality constraints

How to approach it:

- **Step 1:** simplify a weighted constraint $\sum w_i x_i \leq W$ to $\sum x_i \leq k$ if possible, by checking the worst-case sum over subsets of size k (must be $\leq W$) and the best-case sum over subsets of size $k+1$ (must be $> W$).
- **Step 2:** choose an encoding technique: Full Adder (compact, chains carries, *not* arc-consistent in general), bitwise (only for AtMostOne, $O(\log n)$ auxiliary variables), sequential (arc-consistent, $O(nk)$ variables).
- For a Full-Adder encoding: chain $FA_0, FA_1, \dots$ with each carry-out c_{i+1} feeding the next stage's carry-in; always enumerate all input combinations to find the exact extra clause needed (the “carry=1 but sum still allowed” boundary case).
- Always state explicitly which variables of $x_1, \dots, x_n$ are wired to which FA inputs.

Type 1 — Simplification of a weighted constraint (k)

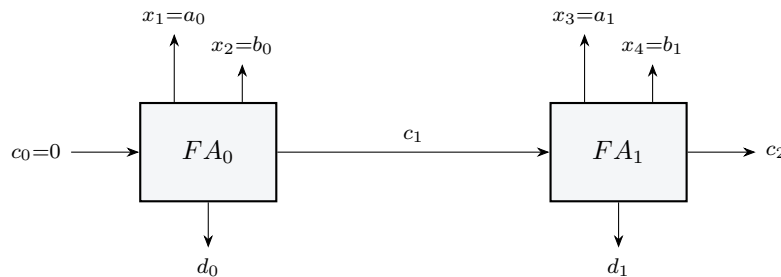
Simplify $5x_1 + 6x_2 + 7x_3 + 8x_4 \leq 17$ to an equivalent constraint $x_1 + x_2 + x_3 + x_4 \leq k$. Give k and justify.

Worst-case pair: $7+8 = 15 \leq 17$ (every pair satisfies). Best-case (lightest) triple: $5+6+7 = 18 > 17$ (every triple violates). Hence exactly 0,1,2 true variables satisfy the constraint and 3,4 violate it: $k = 2$.

Type 2 — Full-Adder cardinality encoding

Encode $x_1 + x_2 + x_3 + x_4 \leq 2$ using the minimum number of Full Adders.

(i) Two chained FAs: $FA_0(a_0, b_0, c_0, d_0, c_1) \rightarrow FA_1(a_1, b_1, c_1, d_1, c_2)$, with FA_0 's carry-out wired into FA_1 's carry-in.



(ii) $a_0 = x_1, b_0 = x_2, a_1 = x_3, b_1 = x_4, c_0 = 0$.

(iii) By enumerating all 16 assignments, $c_2 = 1$ occurs in exactly the cases where the true sum is ≥ 3 , *except* for one boundary case ($x_1 = x_2 = 0, x_3 = x_4 = 1$, sum= 2, where $c_2 = 1$ but

$d_0 = d_1 = c_1 = 0$). The extra clause:

$$\neg c_0 \wedge (c_2 \rightarrow (\neg d_0 \wedge \neg d_1 \wedge \neg c_1))$$

in CNF: $(\neg c_0) \wedge (\neg c_2 \vee \neg d_0) \wedge (\neg c_2 \vee \neg d_1) \wedge (\neg c_2 \vee \neg c_1)$.

Type 3 — AtMostOne — alternative encodings

Encode $\text{AtMostOne}(x_1, x_2, x_3, x_4)$ using (a) the pairwise encoding and (b) the sequential encoding.

(a) **Pairwise:** $\bigwedge_{i < j} (\neg x_i \vee \neg x_j) = \binom{4}{2} = 6$ clauses, arc-consistent, but $O(n^2)$ clauses overall.

(b) **Sequential:** introduce s_1, s_2, s_3 (“at least one true among $x_1, \dots, x_i$ ”):

$$(\neg x_1 \vee s_1) \wedge \bigwedge_{1 < i < 4} [(\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg s_{i-1} \vee \neg x_i)] \wedge (\neg s_3 \vee \neg x_4)$$

Uses $O(n)$ auxiliary variables and clauses, and remains arc-consistent, unlike the Full-Adder encoding.

2.2 Direct SAT Encoding of a CSP

How to approach it:

- This is a different task from cardinality encoding: here the *whole* CSP is translated into SAT, using one Boolean variable per “cell”/decision (e.g. p_{ij} = “value j is assigned to slot i ”), not just a single constraint.
- Systematically translate each CSP constraint type into its propositional counterpart:
 - at least one value per slot i : $\bigvee_j p_{ij}$
 - at most one value per slot i : $\bigwedge_{j < k} \neg(p_{ij} \wedge p_{ik})$
 - together, *exactly one* = at-least-one $\wedge$ at-most-one
- For binary CSP constraints (e.g. two cells cannot both hold a certain pair of values), add one clause $\neg(p_{i_1 j_1} \wedge p_{i_2 j_2})$ per forbidden combination.

Type 1 — N-Queens encoded directly in SAT

Encode the N-Queens problem in SAT using Boolean variables p_{ij} (“there is a queen in row i , column j ”), $1 \leq i, j \leq n$.

At least one queen per row i : $\bigwedge_{1 \leq i \leq n} \bigvee_{1 \leq j \leq n} p_{ij}$

At most one queen per row i : $\bigwedge_{1 \leq i \leq n} \bigwedge_{0 < j < k \leq n} \neg(p_{ij} \wedge p_{ik})$

At most one queen per column (same pattern with i, j swapped): $\bigwedge_{1 \leq j \leq n} \bigwedge_{0 < i < k \leq n} \neg(p_{ij} \wedge p_{kj})$

At most one queen per diagonal (both directions):

$$\bigwedge_{1 \leq i < i' \leq n} \bigwedge_{j, j' : i+j=i'+j' \vee i-j=i'-j'} \neg(p_{ij} \wedge p_{i'j'})$$

Note. Rows already have *exactly one* queen enforced (at-least-one + at-most-one), which combined with at-most-one per column guarantees exactly n queens, one per row and one per column, with no two on the same diagonal — exactly the N-Queens constraints, expressed only with propositional clauses.

2.3 Solving: Resolution and DPLL

How to approach it:

- **Resolution rule:** from $(\ell \vee C_1)$ and $(\neg \ell \vee C_2)$, derive $(C_1 \vee C_2)$. To prove UNSAT, keep resolving until the empty clause $\square$ is derived. Resolution can prove unsatisfiability but does not directly hand you a satisfying assignment.
- **DPLL:** repeat unit propagation (simplify all unit clauses) until no more apply, then branch on an unassigned variable (try ℓ , then $\neg \ell$ on backtrack), recursing; fail only when no branch remains.
- When asked to “simulate DPLL”, track a trail of decided/derived literals explicitly, and always do unit propagation to a fixpoint before making a new decision.

Type 1 — Resolution refutation

Show, via resolution, that $\{(x_1 \vee x_2), (\neg x_1 \vee x_2), (x_1 \vee \neg x_2), (\neg x_1 \vee \neg x_2)\}$ is unsatisfiable.

Resolve $(x_1 \vee x_2)$ and $(\neg x_1 \vee x_2)$ on x_1 : derive (x_2) . Resolve $(x_1 \vee \neg x_2)$ and $(\neg x_1 \vee \neg x_2)$ on x_1 : derive $(\neg x_2)$. Resolve (x_2) and $(\neg x_2)$ on x_2 : derive the empty clause $\square$. Hence the formula is UNSAT.

Type 2 — DPLL simulation

Run DPLL on $\{(x_1 \vee x_2), (\neg x_1 \vee x_3), (\neg x_2 \vee \neg x_3), (x_1)\}$.

Unit clause (x_1) forces $x_1 = \text{T}$. Propagate: from $(\neg x_1 \vee x_3)$, derive $x_3 = \text{T}$ (unit). Propagate $x_3 = \text{T}$ into $(\neg x_2 \vee \neg x_3)$: becomes unit $(\neg x_2)$, forcing $x_2 = \text{F}$. All clauses satisfied ($x_1 \vee x_2$ satisfied by x_1), no conflict $\Rightarrow$ SAT with $x_1 = \text{T}, x_2 = \text{F}, x_3 = \text{T}$. No branching decision was even needed here — unit propagation alone solved it.

3 SMT

3.1 Theories and CDCL(T)

How to approach it:

- Identify *which* background theory the atoms belong to (LRA/LIA, arrays, strings, EUF, bit-vectors) before reasoning — the rules of that theory (not propositional logic) decide satisfiability of a set of theory literals.
- **Lazy SMT / CDCL(T)**: the Boolean skeleton is solved with a SAT-style search (decide, propagate, conflict), while a specialized *theory solver* checks consistency of the theory literals implied by the current partial assignment, feeding T-conflicts back as learned clauses.
- **Purification**: rewrite a mixed formula (several theories) by introducing fresh shared variables so that each literal belongs to a single theory — necessary for theory combination (e.g. Nelson–Oppen).

Type 1 — Theory reasoning (strings)

In the theory of strings, with $+$ denoting concatenation, is $x + y = y + z \wedge x \neq z$ satisfiable?

No. String concatenation is associative and the equation $x + y = y + z$ (together with basic properties of string equality) forces $x = z$ in this simple case — so $x \neq z$ contradicts it. The formula is unsatisfiable.

Type 2 — Purification

Given the mixed formula $f(x) = y \wedge x = y + 1$ where f is an uninterpreted function (EUF) and $+$ belongs to linear arithmetic, purify it into single-theory literals.

Introduce a fresh shared variable z : rewrite as $f(x) = y \wedge x = z \wedge z = y + 1$ where z acts as the shared constant/variable connecting the EUF literal ($f(x) = y$) and the arithmetic literal ($z = y + 1$), so each literal now belongs to exactly one theory — ready for Nelson–Oppen style combination.

Type 3 — Ackermann encoding of EUF (eliminating uninterpreted functions/predicates)

Consider predicates $p(x, y, y)$ and $p(x, z, t)$ in the EUF theory. Remove them by encoding into SAT via the Ackermann reduction.

Replace $p(x, y, y)$ with a fresh Boolean P_1 and $p(x, z, t)$ with a fresh Boolean P_2 . Since p is a function of its arguments, if the two argument tuples are equal, P_1 and P_2 must agree (functional consistency / congruence). Add the congruence axiom:

$$(x = x \wedge y = z \wedge y = t) \rightarrow P_1 = P_2 \quad \text{i.e.} \quad y \neq z \vee y \neq t \vee P_1 = P_2$$

(the trivial $x = x$ conjunct drops out). The resulting formula, with all equalities between arguments encoded as fresh Booleans, is then purely propositional and can be solved by a plain SAT solver

— this is the essence of the “eager” (as opposed to lazy) SMT approach for EUF. The *Bryant* encoding is a variant that instead nests the case distinction using an **ite**: $p(x, z, t) \rightsquigarrow \text{ite}(x=x \wedge y=z \wedge y=t, P_1, P_2)$.

3.2 Theory of Arrays

How to approach it:

- Always start from the three axioms and keep them at hand:

- (i) $(\forall a)(\forall i)(\forall v) \text{read}(\text{write}(a, i, v), i) = v$
- (ii) $(\forall a)(\forall i)(\forall j)(\forall v) i \neq j \rightarrow \text{read}(\text{write}(a, i, v), j) = \text{read}(a, j)$
- (iii) $(\forall a)(\forall a') ((\forall i) \text{read}(a, i) = \text{read}(a', i)) \rightarrow a = a'$ (extensionality)

- To prove unsatisfiability: pick the relevant axiom instance for the terms actually appearing in the formula, and derive a contradiction with an explicit literal in the formula.
- When an index equality (e.g. $i = j$) is unknown, **case-split**: try $i = j$ and $i \neq j$ separately, applying axiom (i) or (ii) respectively in each branch.

Type 1 — Deciding T_A -satisfiability

Is the following formula T_A -satisfiable? $\text{write}(a, i, x) \neq a \wedge \text{read}(a, i) = x$

Let $a' = \text{write}(a, i, x)$. Since $a' \neq a$, by extensionality (iii) there must exist some index where the two arrays disagree: $(\exists j) \text{read}(a', j) \neq \text{read}(a, j)$. Case-split on j :

- If $i \neq j$: axiom (ii) gives $\text{read}(a', j) = \text{read}(a, j)$, contradicting $\text{read}(a', j) \neq \text{read}(a, j)$.
- If $i = j$: then $\text{read}(a', i) \neq \text{read}(a, i)$. Axiom (i) gives $\text{read}(a', i) = x$, so $x \neq \text{read}(a, i)$ — but the formula states $\text{read}(a, i) = x$, a contradiction.

Both cases lead to a contradiction, so the formula is T_A -unsatisfiable.

3.3 Lazy SMT: the abstraction–refinement loop

How to approach it:

- Abstract every theory atom in φ into a fresh Boolean literal, obtaining a propositional skeleton Φ ; solve Φ with a SAT solver to get a candidate model M .
- Check whether M (read back as a set of theory literals) is T -consistent using the theory’s own rules (e.g. congruence for EUF, Simplex for LRA).
- If T -inconsistent, the theory solver returns a *blocking clause* $\neg M$ (the negation of the conflicting literals) which is added to Φ , and the SAT solver is asked for a new model.
- Repeat until either a T -consistent model is found (SAT) or the propositional formula itself becomes UNSAT (UNSAT overall).

Type 1 — Simulating the CDCL(T) abstraction-refinement loop

Consider the EUF formula $\varphi : \underbrace{g(a) = c}_{\ell_1} \wedge \underbrace{(f(g(a)) \neq f(c))}_{\neg \ell_2} \vee \underbrace{g(a) = d}_{\ell_3} \wedge \underbrace{c \neq d}_{\neg \ell_4}$. Simulate the lazy SMT loop.

Abstraction. $\Phi = \{\ell_1, \neg \ell_2 \vee \ell_3, \neg \ell_4\}$.

Round 1. SAT returns $M = \{\ell_1, \neg \ell_2, \neg \ell_4\}$. Theory check: ℓ_1 says $g(a) = c$; by congruence this forces $f(g(a)) = f(c)$, contradicting $\neg \ell_2$ ($f(g(a)) \neq f(c)$) $\Rightarrow T$ -inconsistent. Blocking clause: $\neg \ell_1 \vee \ell_2 \vee \ell_4$ is added: $\Phi' = \Phi \cup \{\neg \ell_1 \vee \ell_2 \vee \ell_4\}$.

Round 2. SAT returns $M' = \{\ell_1, \ell_2, \ell_3, \neg \ell_4\}$. Theory check: ℓ_1 ($g(a) = c$) and ℓ_3 ($g(a) = d$) together force $c = d$ by transitivity, contradicting $\neg \ell_4$ ($c \neq d$) $\Rightarrow T$ -inconsistent again. A new blocking clause is learned and added, and the loop continues until either a T -consistent model is found or the propositional abstraction becomes UNSAT.

3.4 Clause Learning in SAT/SMT (CDCL / CDCL(T))

How to approach it:

- If conflict analysis produces a conflict set $S = \{C_1, \dots, C_n\}$ (the literals responsible for the conflict), the learned clause is their negated disjunction: $\neg C_1 \vee \dots \vee \neg C_n$.
- After learning a clause at decision level K via the 1-UIP scheme, CDCL(T) backjumps to the *second-highest* decision level among the learned clause's literals (not K , not $K-1$ automatically) — this is what lets the solver skip irrelevant parts of the search tree.
- Distinguish clearly between a *decision* (guessed literal, increases decision level) and a *propagation* (forced literal, same decision level as its antecedent).

Type 1 — Learned clause from a conflict set

If conflict analysis produces a conflict set $S = \{C_1, \dots, C_n\}$, what clause is typically learned?

$\neg C_1 \vee \dots \vee \neg C_n$: the clause asserting that at least one of the literals responsible for the conflict must be false, preventing the solver from repeating the same conflicting combination.

Type 2 — Backjump level

After a 1-UIP clause is learned at decision level K , to which level does CDCL(T) backjump?

To the highest decision level among the literals of the learned clause, *excluding* level K itself — i.e., the second-highest decision level appearing in the clause. This may skip several decision levels at once, unlike chronological backtracking.

4 LP / ILP / MIP

4.1 Modelling

How to approach it:

- Structure exactly as for CP: **Variables**, **Constraints**, **Objective**, each commented.
- Linearizing common logical patterns (all with binary $y_i \in \{0, 1\}$):
 - Mutual exclusion (“not both A and B ”): $y_A + y_B \leq 1$.
 - Implication (“if A then B ”): $y_A \leq y_B$.
 - Conjunction bonus / AND variable ($z = y_A \wedge y_B$): $z \leq y_A$, $z \leq y_B$, $z \geq y_A + y_B - 1$, plus benefit term + bonus $\cdot z$ in the objective.
- Only linear constraints/objective are allowed — if the exam says so explicitly, do not use products of two variables directly; introduce an auxiliary linearized variable as above.

Type 1 — MIP formulation with logical constraints

A system can enable three optional features: RAG (benefit 8, GPU cost 3), Code Execution (benefit 10, cost 4), Vision (benefit 12, cost 5). GPU budget is 8. Rules: Code Execution and Vision cannot both be selected; if Vision is selected then RAG must also be selected; if both RAG and Code Execution are selected, an additional benefit of 3 is gained. Formulate a MIP maximizing total benefit.

Variables. $y_R, y_{CE}, y_V \in \{0, 1\}$ (feature selected or not); $z \in \{0, 1\}$, auxiliary variable representing “RAG and Code Execution both selected” ($z = y_R \wedge y_{CE}$).

Constraints.

$$\begin{array}{ll} y_{CE} + y_V \leq 1 & \text{(mutual exclusion)} \\ y_V \leq y_R & \text{(implication: Vision} \Rightarrow \text{RAG)} \\ z \leq y_R, \quad z \leq y_{CE}, \quad z \geq y_R + y_{CE} - 1 & \text{(linearized AND)} \\ 3y_R + 4y_{CE} + 5y_V \leq 8 & \text{(GPU budget)} \end{array}$$

Objective.

$$\max 8y_R + 10y_{CE} + 12y_V + 3z$$

Type 2 — Describing how a MIP solver works

Briefly describe, in natural language, how a modern MIP solver would solve the model above (max. 10 lines).

The solver first relaxes integrality, solving the continuous LP relaxation via the simplex method to obtain an upper bound (for maximization). If the LP optimum is already integral, it is the MIP optimum. Otherwise, it applies *branch-and-bound*: it selects a fractional variable and branches into two subproblems (e.g. $y_i = 0$ / $y_i = 1$), recursively solving the LP relaxation of each; branches

whose LP bound is no better than the best integer solution found so far are pruned. Modern solvers additionally strengthen the relaxation with *cutting planes* (e.g. Gomory cuts, cover cuts, clique cuts) derived from the problem structure, tightening bounds before or during branching. The search continues until every branch is either pruned, infeasible, or yields an integral solution, and the best integral solution found is returned as optimal.

4.2 Simplex, Gomory Cuts, and Branch-and-Bound (numeric)

How to approach it:

- **Simplex step:** put the LP in standard form (add slack variables), pick an entering variable (most attractive reduced cost), run the ratio test on the RHS/coefficient to find the leaving variable, pivot.
- **Reading off the bound:** after pivoting, the current objective value is a valid *upper* bound (for a maximization problem) on the true integer optimum, since the LP relaxation always dominates the ILP.
- **Gomory cut:** for a basic variable $x_k^* = \beta_k + \sum_j \alpha_{k,j}x_j$ with fractional β_k , take fractional parts $f_k = \beta_k - \lfloor \beta_k \rfloor$ and $f_{k,j}$ of the coefficients; the cut is $\sum_j f_{k,j}x_j \geq f_k$ (equivalently written as $-f_k + \sum_j f_{k,j}x_j \geq 0$).
- **Branch-and-bound decision:** if the current LP bound is *no better* than the best integer solution found so far (the incumbent), prune the branch; otherwise keep branching (split on a fractional variable, e.g. $x_i \leq \lfloor x_i^* \rfloor$ vs. $x_i \geq \lceil x_i^* \rceil$).

Type 1 — Full worked exercise: pivot, cut, and B&B decision

Consider $\max z = 3x_1 + x_2$ s.t. $2x_1 + x_2 \leq 5$, $x_1 + 3x_2 \leq 9$, $x_1, x_2 \geq 0$, $x_1, x_2 \in \mathbb{Z}$. **(1)** Reduce to standard form and apply one simplex pivot to the LP relaxation; give the basic solution and an upper bound on z . **(2)** Define the Gomory cut for the variable with fractional value. **(3)** Given an incumbent bound $z \geq k$, what is the next B&B step if $k < 7$? If $k = 7$?

(1) Standard form: $2x_1 + x_2 + s_1 = 5$, $x_1 + 3x_2 + s_2 = 9$, $s_1, s_2 \geq 0$. Start from $x_1 = x_2 = 0$, $s_1 = 5$, $s_2 = 9$ ($z = 0$).

Entering variable: x_1 (highest coefficient, 3). Ratio test: $5/2 = 2.5$ vs. $9/1 = 9 \Rightarrow s_1$ leaves. Pivoting on row 1 gives:

$$x_1 + 0.5x_2 + 0.5s_1 = 2.5 \quad 2.5x_2 - 0.5s_1 + s_2 = 6.5$$

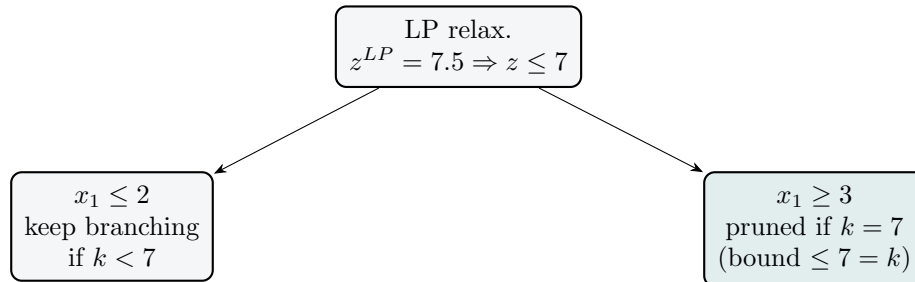
Basic solution: $x_1 = 2.5$, $s_2 = 6.5$, $x_2 = s_1 = 0$. Substituting into the objective: $z = 7.5 - 0.5x_2 - 1.5s_1$ — both reduced costs are ≤ 0 , so this is already the LP-relaxation optimum after a single pivot: $z^{LP} = 7.5$. Since z must be integer, this gives the upper bound $z \leq 7$.

(2) Gomory cut on x_1 (fractional, $\beta = 2.5$, $f = 0.5$), from the row $x_1 + 0.5x_2 + 0.5s_1 = 2.5$ (coefficients already in $[0, 1]$):

$$0.5x_2 + 0.5s_1 \geq 0.5 \quad \text{i.e.} \quad x_2 + s_1 \geq 1$$

(3) The LP bound tells us no integer solution can exceed $z = 7$.

- If $k < 7$ (e.g. $k = 6$): the bound (7) is still strictly better than the incumbent, so pruning is *not* justified — continue branching (e.g. on $x_1 \leq 2$ vs. $x_1 \geq 3$).
- If $k = 7$: the incumbent already matches the best possible bound, so this branch can be **pruned** — no better integer solution can exist here.



4.3 Duality and Sensitivity

How to approach it:

- **Weak duality:** for a primal maximization and its dual minimization, every dual feasible value is an upper bound on every primal feasible value (and vice versa framed the other way).
- **Sensitivity analysis:** studies how perturbing the input data (costs, right-hand sides) affects feasibility or optimality of the current solution — not about enumerating integer solutions or partitioning the feasible region.
- Infeasibility of the primal does *not* generally correspond one-to-one with infeasibility of the dual (be careful with this common quiz trap: primal infeasible can mean dual unbounded or infeasible, depending on the case).

Type 1 — Primal/dual bound relation (quiz-style)

Consider a primal maximization problem and its dual minimization problem. Which statement is correct? (a) primal infeasibility $\equiv$ dual infeasibility; (b) primal and dual feasible values always coincide; (c) a primal feasible value is an upper bound on a dual feasible value; (d) a dual feasible value is an upper bound on a primal feasible value.

(d). By weak duality, every dual-feasible objective value upper-bounds every primal-feasible objective value in a max/min primal/dual pair.